Shaping Latent World Models via Latent Reasoning for Embodied Planning and Generalization

1 Introduction

We study action-conditioned world modeling for embodied agents. Given an observation image o_t and an action a_t, the goal is to predict the next world state o_{t+1}. Our core hypothesis is that a *structured latent token block* can serve as a unified state carrier that fuses vision, action-conditioning, and future prediction. Concretely, we train a multimodal LLM/VLM to generate a fixed-length latent block $\hat{z}_{t+1}$ that represents the next state, and then ground $\hat{z}_{t+1}$ to real transitions using simulator trajectories and image-level supervision. We refer to Fig. 1 for the data collection and training stages.

Compared with purely pixel-space predictors or opaque latent embeddings, our approach explicitly exposes an intermediate state variable (the latent block) that is (i) easy to iterate for multi-step rollouts and (ii) directly evaluable via decoded next-frame prediction. Recent latent world-model and interactive environment work motivates scalable transition learning in latent spaces [2, 7, 1]. The resulting interface is:

$$(o_t, a_t) \xrightarrow{f_\theta} \hat{z}_{t+1} \xrightarrow{g_\phi} \hat{o}_{t+1}.$$

where f_θ is an LLM/VLM-based latent transition model and g_ϕ is a diffusion-based image decoder.

2 Problem Formulation

We consider a POMDP $\mathcal{M} = (\mathcal{O}, \mathcal{A}, T, R)$ where $o_t \in \mathcal{O}$ is an RGB observation, $a_t \in \mathcal{A}$ is an action (discrete or continuous), and $T(o_{t+1} \mid o_t, a_t)$ defines the transition distribution. We assume access to a dataset of transition tuples:

$$\mathcal{D} = \{(o_t, a_t, o_{t+1})\}_{t=1}^{N}$$

collected from a simulator (e.g., SimWorld) under behavior policies [6].

Latent state carrier. We define a latent state block $\hat{z}_{t+1} \in \mathbb{R}^{K \times d}$ represented as K latent tokens of dimension d, serialized with explicit delimiters:

$$\hat{z}_{t+1} = \langle \texttt{latent_start} \rangle \; z^{(1)} \; \ldots \; z^{(K)} \; \langle \texttt{latent_end} \rangle.$$

The latent transition model f_θ predicts $\hat{z}_{t+1}$ conditioned on (o_t, a_t):

$$\hat{z}_{t+1} = f_\theta(o_t, a_t).$$

A diffusion decoder g_ϕ maps $\hat{z}_{t+1}$ to the next observation prediction:

$$\hat{o}_{t+1} = g_\phi(\hat{z}_{t+1}).$$

We train (θ, ϕ) such that decoded predictions match ground-truth transitions.

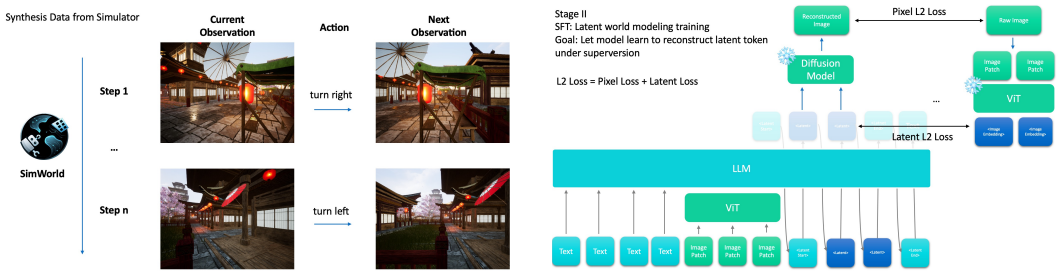
3 Method

3.1 Model Components

Vision encoder. A ViT encoder E_v maps o_t to patch tokens:

$$h_t = E_v(o_t) \in \mathbb{R}^{M \times d_v}.$$

Action encoding. We encode a_t into tokens $u_t = \mathrm{Enc}(a_t)$. For discrete actions, Enc is an embedding lookup; for continuous actions, it can be an MLP projection.



(a) SimWorld data collection (tuples o_t, a_t, o_{t+1}).

(b) Stage II training: decode $\hat{z}_{t+1}$ to $\hat{o}_{t+1}$ and apply pixel/latent losses.

Figure 1: Overview of data collection in SimWorld (left) and Stage II world-model supervision (right).

Latent transition model. A multimodal LLM/VLM f_θ consumes (h_t, u_t) and emits a fixed-length latent block:

$$\hat{z}_{t+1} = f_\theta(h_t, u_t).$$

Importantly, $\hat{z}_{t+1}$ is not an explanation; it is trained as a predictive state variable.

Diffusion decoder. We use a conditional diffusion model g_ϕ that generates an image conditioned on the latent block. Conceptually, we treat $\hat{z}_{t+1}$ as the conditioning input c:

$$\hat{o}_{t+1} = g_\phi(c = \hat{z}_{t+1}).$$

In practice, we implement conditioning via cross-attention from U-Net blocks to a latent-context sequence derived from $\hat{z}_{t+1}$ (e.g., a learned projection to match the diffusion text-conditioning dimension).

3.2 Training Pipeline Overview

We adopt a three-stage training pipeline:

1. **Stage 0 (Diffusion Adaptation):** Fine-tune the diffusion decoder g_ϕ on the target transition dataset (next-frame images o_{t+1}) so it can faithfully reconstruct in-domain observations.
2. **Stage I (Latent Block Stabilization):** Supervised fine-tune f_θ to reliably emit well-formed $\langle \texttt{latent_start} \rangle \ldots \langle \texttt{latent_end} \rangle$ blocks with fixed length K.
3. **Stage II (Transition Grounding):** Train f_θ (and optionally parts of g_ϕ) such that $\hat{z}_{t+1}$ decodes through the (adapted) diffusion model to match o_{t+1}.

This separation is motivated by practicality: diffusion decoders are sensitive to domain shift, and grounding latent blocks through a poorly adapted decoder yields noisy gradients and unstable training.

We will evaluate transition prediction quality using recent embodied benchmarks and evaluation suites, including BEHAVIOR-1K [5], EmbodiedEval [3], and EWMBench [4].

3.3 Stage 0: Fine-tuning the Diffusion Decoder on Task Data

We fine-tune a conditional diffusion model to reconstruct o_{t+1} in-domain. Let $x = o_{t+1}$ be the target image. We use the standard diffusion objective:

$$\mathcal{L}_{\text{diff}}(\phi) = \mathbb{E}_{x \sim \mathcal{D},\, \epsilon \sim \mathcal{N}(0,I),\, t \sim \mathcal{U}[1,T]} \left[\|\epsilon - \epsilon_\phi(x_t, t, c)\|_2^2 \right],$$

where x_t is the noised image at timestep t, and c is the conditioning. During Stage 0, we set c to a *proxy conditioning* derived from available signals (e.g., text prompt describing the scene, or simply use unconditional fine-tuning if no conditioning is needed). The key goal is to adapt g_ϕ so that later, when conditioned on latent blocks, it can reconstruct realistic o_{t+1} in the same domain.

Outputs of Stage 0. After Stage 0, we obtain an in-domain diffusion decoder $g_{\phi^\star}$ used as the reconstruction engine in Stage II. We will typically freeze most of $g_{\phi^\star}$ during Stage II (or allow light LoRA-style adaptation) to stabilize training.

3.4 Stage I: Latent Block Formatting and Stabilization

We fine-tune f_θ to generate well-formed latent blocks with fixed length K and explicit delimiters. We construct training targets that include delimiters and placeholder latent tokens:

$$y_t = \langle \texttt{latent_start} \rangle z^{(1)} \dots z^{(K)} \langle \texttt{latent_end} \rangle.$$

We optimize a token-level cross-entropy objective:

$$\mathcal{L}_{\text{format}}(\theta) = -\sum_{i=1}^{K} \log p_\theta\Big(z^{(i)} \mid o_t, a_t, z^{(<i)}\Big) \; - \; \log p_\theta(\texttt{end} \mid \cdot).$$

This stage focuses on *channel reliability*: correct start/end markers, bounded length, and stable emission.

3.5 Stage II: Grounding Latent Transitions via Diffusion Reconstruction

Given a tuple (o_t, a_t, o_{t+1}), we predict a latent block:

$$\hat{z}_{t+1} = f_\theta(o_t, a_t),$$

then reconstruct:

$$\hat{o}_{t+1} = g_{\phi^\star}(\hat{z}_{t+1}).$$

Reconstruction loss. We use pixel loss plus a semantic embedding loss. Let E_v be a ViT feature extractor:

$$\mathcal{L}_{\text{pixel}} = \|\hat{o}_{t+1} - o_{t+1}\|_2^2, \qquad \mathcal{L}_{\text{embed}} = \|E_v(\hat{o}_{t+1}) - E_v(o_{t+1})\|_2^2.$$

Total grounding loss:

$$\mathcal{L}_{\text{ground}}(\theta) = \mathcal{L}_{\text{pixel}} + \lambda\, \mathcal{L}_{\text{embed}}.$$

We primarily update θ (the latent transition model) while freezing $\phi^\star$ to reduce instability. We will also test partial unfreezing (e.g., last U-Net blocks or LoRA) as an ablation.

Training signal path. Gradients from $\mathcal{L}_{\text{ground}}$ flow through the diffusion decoder into the latent block, thereby shaping $\hat{z}_{t+1}$ to become a predictive state variable aligned with true transitions.

3.6 Multi-step Rollout and Consistency

After training, we can roll out multiple steps under an action sequence $\{a_t, \dots, a_{t+H-1}\}$:

$$\hat{z}_{t+1} = f_\theta(o_t, a_t), \quad \hat{o}_{t+1} = g_{\phi^\star}(\hat{z}_{t+1}),$$
$$\hat{z}_{t+2} = f_\theta(\hat{o}_{t+1}, a_{t+1}), \quad \hat{o}_{t+2} = g_{\phi^\star}(\hat{z}_{t+2}), \; \dots$$

We quantify drift using per-step pixel/embedding error under fixed action sequences:

$$\text{Drift}_H = \frac{1}{H} \sum_{k=1}^{H} \left(\|\hat{o}_{t+k} - o_{t+k}\|_2 + \alpha \|E_v(\hat{o}_{t+k}) - E_v(o_{t+k})\|_2 \right).$$

This directly tests whether the latent state carrier supports stable long-horizon prediction.

4 Theoretical Perspective

Our method can be interpreted as learning an approximate latent sufficient statistic for predicting the next observation under action-conditioning. If the latent block $\hat{z}_{t+1}$ captures a predictive representation of the transition, then there exists a decoder g such that:

$$o_{t+1} \approx g(\hat{z}_{t+1}), \qquad \hat{z}_{t+1} = f(o_t, a_t).$$

However, multi-step rollouts suffer from compounding error because the model conditions on its own predictions:

$$\hat{o}_{t+1} \neq o_{t+1} \;\Rightarrow\; \hat{z}_{t+2} = f(\hat{o}_{t+1}, a_{t+1})$$

is evaluated on shifted inputs. This motivates: (i) semantic losses ($\mathcal{L}_{\text{embed}}$) to reduce distribution mismatch in feature space, and (ii) explicit multi-step evaluation and potential future multi-step training objectives.

5 Experiments

5.1 Datasets and Data Collection

We use simulator-collected transition tuples (o_t, a_t, o_{t+1}) as the primary training source. We will construct datasets across multiple environments/scenes to evaluate both in-domain prediction and generalization. Key dataset axes include:

- **Scene diversity:** training scenes vs. held-out scenes,
- **Action diversity:** behavior-policy actions vs. shifted action distributions,
- **Task context:** optional language context (if available) describing goals or high-level commands.

To enable reproducibility, we will release the data-generation scripts/configs (where permitted) and report the exact simulator version, camera parameters, and action discretization used in collection.

5.2 Training

Our training pipeline consists of three sequential stages: (i) domain adaptation of the diffusion decoder, (ii) latent-channel stabilization, and (iii) transition grounding.

Stage 0: Diffusion domain adaptation. We first adapt the diffusion decoder to the distribution of simulator-generated next-frame observations. This step reduces domain shift between the pretrained diffusion backbone and our task-specific visual data. We evaluate both unconditional adaptation and conditioned adaptation (when proxy context is available). Unless otherwise specified, the adapted diffusion model is frozen during later stages to provide a stable reconstruction interface.

Stage I: Latent channel stabilization. Before grounding latent states to environment transitions, we fine-tune the transition model to reliably emit well-formed fixed-length latent blocks with correct delimiters. This stage isolates syntactic reliability from predictive accuracy. We monitor formatting success rate and latent-length variance to ensure stable channel behavior prior to reconstruction supervision.

Stage II: Transition grounding. Using transition tuples (o_t, a_t, o_{t+1}), we train the latent transition model by supervising reconstructed next frames. Gradients are primarily propagated to the latent transition model, while the diffusion decoder remains frozen unless stated otherwise. We additionally evaluate partial adaptation strategies (e.g., lightweight fine-tuning) to quantify sensitivity to decoder plasticity.

5.3 Evaluation

We evaluate predictive quality along two axes: single-step fidelity and multi-step stability.

One-step prediction. For single-step transitions, we report: (i) pixel-space reconstruction error (MSE / PSNR), and (ii) feature-space discrepancy using a fixed vision encoder. These metrics measure both low-level visual accuracy and higher-level semantic alignment.

Multi-step rollout stability. To assess compounding error, we perform iterative rollouts under fixed action sequences for horizons up to $H \in \{5, 10, 20\}$. We report average rollout drift relative to ground-truth trajectories. We further evaluate robustness under small perturbations of action sequences to probe the stability of learned latent dynamics.

Ablation studies. We conduct controlled ablations to isolate component contributions:

- Removing diffusion adaptation (Stage 0),
- Varying latent block length K,
- Sweeping loss trade-off coefficients,
- Freezing versus partially adapting the diffusion decoder.

These experiments characterize both representational capacity and long-horizon predictive consistency.

5.4 Expected Findings

We expect diffusion adaptation (Stage 0) to be essential for stable grounding: without it, decoded images may be out-of-domain, producing noisy gradients that prevent $\hat{z}_{t+1}$ from becoming a reliable state. We also expect embedding losses to reduce drift over long horizons by encouraging semantic consistency.

6 Discussion

6.1 Novelty

Latent token block as explicit next-state: we treat an LLM-emitted latent block as the state variable for action-conditioned transitions; **Decoder-first grounding:** we explicitly fine-tune the diffusion decoder to the task domain before using it to supervise latent-state learning; **Two-stage latent stabilization and grounding:** separate objectives for (i) reliable latent channel emission and (ii) transition alignment.

6.2 Advantages

Interpretable interface for rollouts: latent blocks can be iterated for multi-step prediction; **Flexible supervision:** combine pixel and semantic alignment; decoder provides strong visual grounding; **Compatibility with LLM planning:** latent rollouts can integrate with System-II planners without forcing chain-of-thought explanations.

6.3 Limitations

Compute cost: diffusion decoding is expensive for long-horizon rollouts; **Reconstruction bias:** pixel losses may overemphasize appearance instead of abstract dynamics; **Compounding error:** distribution shift under self-generated rollouts remains challenging.

Potential future work includes multi-step training objectives (teacher-forcing vs. scheduled sampling), contrastive latent alignment, and planning-aware losses that optimize downstream control performance.

References

[1] Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rockt"aschel. Genie: Generative interactive environments. *arXiv preprint arXiv:2402.15391*, 2024. URL `https://arxiv.org/abs/2402.15391`.

[2] Maxime Burchi and Radu Timofte. Mudreamer: Learning predictive world models without reconstruction. *arXiv preprint arXiv:2405.15083*, 2024. URL `https://arxiv.org/abs/2405.15083`.

[3] Zhili Cheng, Yuge Tu, Ran Li, Shiqi Dai, Jinyi Hu, Shengding Hu, Jiahao Li, Yang Shi, Tianyu Yu, Weize Chen, Lei Shi, and Maosong Sun. Embodiedeval: Evaluate multimodal llms as embodied agents. *arXiv preprint arXiv:2501.11858*, 2025. URL `https://arxiv.org/abs/2501.11858`.

[4] Yue Hu, Siyuan Huang, Yue Liao, Shengcong Chen, Pengfei Zhou, Liliang Chen, Maoqing Yao, and Guanghui Ren. Ewmbench: Evaluating scene, motion, and semantic quality in embodied world models. *arXiv preprint arXiv:2505.09694*, 2025. URL `https://arxiv.org/abs/2505.09694`.

[5] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martin-Martin, Chen Wang, Gabrael Levine, Wensi Ai, Benjamin Martinez, Hang Yin, Michael Lingelbach, Minjune Hwang, Ayano Hiranaka, Sujay Garlanka, Arman Aydin, Sharon Lee, Jiankai Sun, Mona Anvari, Manasi Sharma, Dhruva Bansal, Samuel Hunter, Kyu-Young Kim, Alan Lou,

Caleb R. Matthews, Ivan Villa-Renteria, Jerry Huayang Tang, Claire Tang, Fei Xia, Yunzhu Li, Silvio Savarese, Hyowon Gweon, C. Karen Liu, Jiajun Wu, and Li Fei-Fei. Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*, 2024. URL https://arxiv.org/abs/2403.09227.

[6] Jiawei Ren, Yan Zhuang, Xiaokang Ye, Lingjun Mao, Xuhong He, Jianzhi Shen, Mrinaal Dogra, Yiming Liang, Ruixuan Zhang, Tianai Yue, Yiqing Yang, Eric Liu, Ryan Wu, Kevin Benavente, Rajiv Mandya Nagaraju, Muhammad Faayez, Xiyan Zhang, Dhruv Vivek Sharma, Xianrui Zhong, Ziqiao Ma, Tianmin Shu, Zhiting Hu, and Lianhui Qin. Simworld: An open-ended realistic simulator for autonomous agents in physical and social worlds. *arXiv preprint arXiv:2512.01078*, 2025. URL https://arxiv.org/abs/2512.01078.

[7] Siyuan Zhou, Yilun Du, Jiaben Chen, Yandong Li, Dit-Yan Yeung, and Chuang Gan. Robodreamer: Learning compositional world models for robot imagination. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 61885–61896, 2024. URL https://proceedings.mlr.press/v235/zhou24f.html.